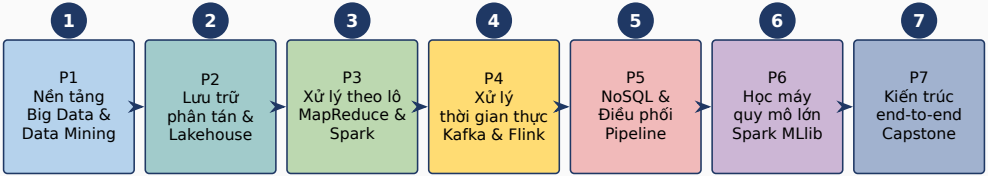
THỰC HÀNH: LẬP TRÌNH SPARK

Biến lý thuyết RDD, DataFrame và Spark SQL thành chương trình chạy được (2 tiết)



Mục tiêu thực hành Tuần 4

- Chạy được chương trình WordCount bằng PySpark theo phong cách RDD, có dùng `cache()`
- Đọc dữ liệu dạng bảng (CSV) và thực hiện biến đổi/tổng hợp bằng DataFrame API
- Viết truy vấn tương đương bằng Spark SQL, thử Window Function và một Broadcast Join
- Quan sát Spark UI để hiểu cách một job được chia thành Stage/Task, và nhận diện data skew

Bước 1 — Khởi động PySpark & kiểm tra môi trường

```
# Môi trường Spark đã cài ở Tuần 1 (Bước 7) -- chỉ cần khởi động lại
# Cách 1: dùng PySpark shell tương tác
pyspark

# Cách 2: viết script rồi chạy bằng spark-submit
spark-submit bai_tap_tuan4.py

# Kiểm tra Spark UI (mặc định) tại:
# http://localhost:4040
```

Bước 2 — WordCount thực tế bằng RDD API (có cache)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Tuan4-WordCount") \
    .master("local[*]") \
    .getOrCreate()
sc = spark.sparkContext

text = sc.textFile("sample.txt").cache()
counts = (text.flatMap(lambda line: line.split())
              .map(lambda w: (w.lower(), 1))
              .reduceByKey(lambda a, b: a + b))

for word, cnt in counts.takeOrdered(10, key=lambda x: -x[1]):
    print(word, cnt)
```

Bước 3 — Đọc dữ liệu bảng bằng DataFrame API

```
df = spark.read.csv("sales.csv", header=True, inferSchema=True)

df.printSchema()
df.show(5)

print("So dong du lieu:", df.count())
```

Bước 4 — Biến đổi & tổng hợp với DataFrame

```
from pyspark.sql import functions as F

(df.filter(df.quantity > 0)
   .groupBy("category")
   .agg(F.sum("revenue").alias("total_revenue"),
        F.count("*").alias("num_orders"))
   .orderBy(F.desc("total_revenue"))
   .show())
```

Bước 5 — Truy vấn tương đương bằng Spark SQL

```
df.createOrReplaceTempView("sales")

result = spark.sql("""
    SELECT category,
           SUM(revenue) AS total_revenue,
           COUNT(*)     AS num_orders
    FROM sales
    WHERE quantity > 0
    GROUP BY category
    ORDER BY total_revenue DESC
""")
result.show()
```

Bước 6 — Thử Window Function: xếp hạng sản phẩm bán chạy

```
from pyspark.sql.window import Window

cua_so = Window.partitionBy("category").orderBy(F.desc("quantity"))

df.withColumn("hang", F.rank().over(cua_so)) \
  .filter("hang <= 3") \
  .show()   # Top 3 san pham ban chay nhat moi danh muc
```

Bước 7 — Thử Broadcast Join với bảng nhỏ

```
danh_muc = spark.createDataFrame(
    [("DienThoai", "Dien tu"), ("Laptop", "Dien tu"), ("AoThun", "Thoi trang")],
    ["category", "nhom_lon"]
)

ket_hop = df.join(F.broadcast(danh_muc), "category", "left")
ket_hop.explain()   # kiem tra trong ke hoach co "BroadcastHashJoin" khong
```

So sánh khi thực hành: RDD API vs. DataFrame/Spark SQL

Tiêu chí	RDD API	DataFrame / Spark SQL
Mức trừu tượng	Thấp — thao tác trực tiếp trên từng phần tử	Cao — thao tác trên bảng có schema
Tối ưu hóa	Không qua Catalyst	Catalyst + Tungsten + AQE tự động tối ưu
Cú pháp	Hàm bậc cao kiểu functional (map/reduce)	Giống SQL hoặc pandas
Phù hợp nhất	Dữ liệu phi cấu trúc, logic tùy biến phức tạp	Dữ liệu có cấu trúc, ETL, phân tích chuẩn

Quan sát Spark UI khi chạy job

- Mỗi `SparkSession` tự khởi chạy một Web UI (mặc định `http://localhost:4040`)
- Tab **Jobs** → **Stages** → **Tasks**: quan sát cách DAG Scheduler chia nhỏ công việc thành các Stage tại mỗi điểm *shuffle* (ví dụ `reduceByKey`, `groupBy`)
- Kiểm tra ở Bước 7 xem kế hoạch thực thi có dùng **BroadcastHashJoin** như mong đợi không (qua `explain()`)
- So sánh thời gian thực thi của job RDD (Bước 2) và job DataFrame (Bước 4) trên cùng một máy để cảm nhận hiệu quả của Catalyst Optimizer

Bài tập thực hành (làm tại lớp / hoàn thành ở nhà)

- Cho tập dữ liệu bán hàng mẫu `sales.csv` (cột: `category`, `product`, `quantity`, `revenue`, `order_date`)
- Viết chương trình PySpark (RDD hoặc DataFrame, tự chọn) trả lời:
  - Tổng doanh thu theo từng `category`, sắp xếp giảm dần
  - Top 5 sản phẩm có tổng `quantity` bán ra cao nhất (gợi ý: dùng Window Function)
  - Tháng nào trong năm có doanh thu cao nhất
- Sản phẩm nộp: file mã nguồn `.py` kèm ảnh chụp kết quả và ảnh chụp Spark UI của job đã chạy

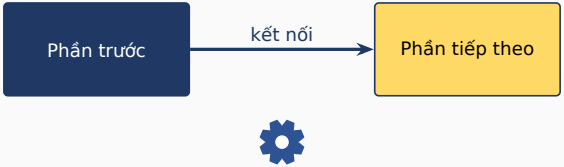
Checklist chuẩn bị trước Tuần 5

- ☐ Đã chạy thành công cả WordCount (RDD) và truy vấn tổng hợp (DataFrame/SQL)
- ☐ Đã mở và quan sát Spark UI ít nhất một lần khi job đang chạy, xác định được Stage nào có shuffle
- ☐ Đã thử Window Function và kiểm tra Broadcast Join bằng `explain()`
- ☐ Đã hoàn thành bài tập thực hành với dữ liệu `sales.csv`
- ☐ Ôn lại khái niệm Lazy Evaluation, Narrow/Wide Dependency, Transformation vs. Action
- ☐ Đọc trước kiến trúc Producer/Consumer/Topic của Apache Kafka cho Tuần 5

Tổng kết Tuần 4

- Lý thuyết — MapReduce
  - Mô hình Map–Shuffle–Reduce; vai trò Combiner, Partitioner; chạy trên YARN; chịu lỗi qua task retry/speculative execution
- Lý thuyết — Apache Spark
  - RDD, lineage, Narrow/Wide Dependency, Shuffle nội bộ, Broadcast/Accumulator
  - DataFrame, Spark SQL, Catalyst (4 giai đoạn), Tungsten, AQE, chiến lược Join, Window Function
  - Data Skew, đọc Spark UI, case study ETL end-to-end
- Thực hành
  - Đã viết và chạy WordCount (RDD), truy vấn tổng hợp (DataFrame/Spark SQL), Window Function và Broadcast Join
  - Đã quan sát Spark UI để hiểu cách một job được chia thành Stage/Task

Cầu nối sang Tuần 5 — Phần 4



*MapReduce và Spark (ở chế độ mặc định) xử lý dữ liệu **đã có sẵn** theo lô. Nhiều bài toán (gian lận thẻ tín dụng, cảnh báo IoT) không thể chờ job batch tiếp theo:*

- Tuần 5 giới thiệu Apache Kafka (hàng đợi sự kiện phân tán) và Spark Structured Streaming / Flink cho xử lý thời gian thực
- Toàn bộ DataFrame API, Catalyst, AQE vừa học ở Tuần 4 sẽ quay lại trực tiếp trong Spark Structured Streaming

Hỏi & Đáp



Hẹn gặp lại ở Tuần 5 — Phần 4: Xử lý dữ liệu thời gian thực

Tài liệu tham khảo: Dean & Ghemawat, "MapReduce" (OSDI 2004); Zaharia et al., "Resilient Distributed Datasets" (NSDI 2012); Apache Spark Documentation